



Reinforcement Learning

P Satya Jayadev

Web Enabled M.Tech,
Indian Institute of Technology Madras

Topic 7

Selected Deep RL Methods and Extension

DQN, DDPG, Decision Transformers and Multi-Agent RL



1. Deep Q-Networks (DQN)

- Stable deep value learning for discrete actions

2. Deep Deterministic Policy Gradient (DDPG)

- Off-policy actor–critic learning for continuous actions

3. Decision Transformers (DT)

- Offline RL as return-conditioned sequence modelling

4. Multi-Agent Reinforcement Learning

- Learning when multiple agents interact in a shared environment



Deep Q-Network (DQN)

DQN Replaces the Q-Table, Not Q-Learning

- ▶ Recall the one-step Q-learning target: $G_t = R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a')$.
- ▶ The tabular update is: $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [G_t - Q(S_t, A_t)]$
- ▶ A Q-table becomes impractical when the state is high-dimensional or continuous.
- ▶ DQN replaces the table with a neural-network approximation: $Q(S, A) \approx Q_w(S, A)$
- ▶ The greedy action is selected using

$$A_t = \arg \max_{a \in \mathcal{A}} Q_w(S_t, a),$$

with occasional exploratory actions during training.

Scope of DQN

DQN handles high-dimensional state representations, but generally requires a manageable discrete action set.



A Neural Q-Function Creates Two Stability Problems

A direct implementation could train Q_w after every transition using

$$G_t(\mathbf{w}) = R_{t+1} + \gamma \max_{a'} Q_w(S_{t+1}, a').$$

However, this creates two major difficulties.

1. **Correlated Training Samples:** Consecutive transitions usually contain similar states and training on them in sequence can bias successive updates.
2. **Moving Learning Targets:** The same network determines both target and observed values. in similar directions.



A Neural Q-Function Creates Two Stability Problems

A direct implementation could train Q_w after every transition using

$$G_t(\mathbf{w}) = R_{t+1} + \gamma \max_{a'} Q_w(S_{t+1}, a').$$

However, this creates two major difficulties.

1. **Correlated Training Samples:** Consecutive transitions usually contain similar states and training on them in sequence can bias successive updates.
2. **Moving Learning Targets:** The same network determines both target and observed values. in similar directions.

Consequence

Updates may oscillate or diverge because the network is learning from strongly related samples while simultaneously changing its own target.



Experience Replay Breaks Temporal Correlation

Instead of training immediately on each consecutive transition, DQN stores experience in a replay buffer:

$$\mathcal{D} = \{(S_j, A_j, R_{j+1}, S_{j+1}, d_j)\},$$

where $d_j = 1$ indicates that the episode ended after the transition.



Experience Replay Breaks Temporal Correlation

Instead of training immediately on each consecutive transition, DQN stores experience in a replay buffer:

$$\mathcal{D} = \{(S_j, A_j, R_{j+1}, S_{j+1}, d_j)\},$$

where $d_j = 1$ indicates that the episode ended after the transition.

1. The agent interacts with the environment and adds new transitions to $\mathcal{D}$.
2. A random minibatch is sampled: $\mathcal{B}_m \sim \mathcal{D}$
3. The Q-network is updated using transitions collected at different times and from different episodes.



Experience Replay Breaks Temporal Correlation

Instead of training immediately on each consecutive transition, DQN stores experience in a replay buffer:

$$\mathcal{D} = \{(S_j, A_j, R_{j+1}, S_{j+1}, d_j)\},$$

where $d_j = 1$ indicates that the episode ended after the transition.

1. The agent interacts with the environment and adds new transitions to $\mathcal{D}$.
2. A random minibatch is sampled: $\mathcal{B}_m \sim \mathcal{D}$
3. The Q-network is updated using transitions collected at different times and from different episodes.

Why Does It Help?

- ▶ Random sampling reduces correlation between consecutive updates.
- ▶ Each transition can contribute to multiple parameter updates.
- ▶ The minibatch contains a mixture of successes, failures and different regions of the state space.
- ▶ It is an off-policy learning and can learn from older transitions as well.



A Target Network Stabilises the TD Target

DQN maintains two Q-networks with the same architecture:

- ▶ **Online network** Q_w : updated after every training minibatch.
- ▶ **Target network** Q_w^t : held fixed temporarily while TD targets are computed.



A Target Network Stabilises the TD Target

DQN maintains two Q-networks with the same architecture:

- ▶ **Online network** Q_w : updated after every training minibatch.
- ▶ **Target network** Q_w^t : held fixed temporarily while TD targets are computed.

For transition j , the target is

$$Y_j = R_{j+1} + \gamma(1 - d_j) \max_{a'} Q_w^t(S_{j+1}, a')$$

- ▶ The online network predicts $Q_w(S_j, A_j)$.
- ▶ The target network provides the bootstrap value for Y_j .



A Target Network Stabilises the TD Target

DQN maintains two Q-networks with the same architecture:

- ▶ **Online network** Q_w : updated after every training minibatch.
- ▶ **Target network** Q_w^t : held fixed temporarily while TD targets are computed.

For transition j , the target is

$$Y_j = R_{j+1} + \gamma(1 - d_j) \max_{a'} Q_w^t(S_{j+1}, a')$$

- ▶ The online network predicts $Q_w(S_j, A_j)$.
- ▶ The target network provides the bootstrap value for Y_j .
- ▶ Since target w changes more slowly, the learning target is more stable.
- ▶ The target parameters may be updated periodically or gradually.



One DQN Minibatch Update

Sample a minibatch $\mathcal{B}_m$ containing b transitions from the replay buffer.

1. Compute one target for every transition:

$$Y_j = R_{j+1} + \gamma(1 - d_j) \max_{a'} Q_{\mathbf{w}}^t(S_{j+1}, a'), \quad j \in \mathcal{B}_m.$$



One DQN Minibatch Update

Sample a minibatch $\mathcal{B}_m$ containing b transitions from the replay buffer.

1. Compute one target for every transition:

$$Y_j = R_{j+1} + \gamma(1 - d_j) \max_{a'} Q_{\mathbf{w}}^t(S_{j+1}, a'), \quad j \in \mathcal{B}_m.$$

2. Get q-value predictions from the online network for every (S_j, A_j) in the batch: $Q_{\mathbf{w}}(S_j, A_j)$
3. Average the squared TD errors:

$$L_m(\mathbf{w}) = \frac{1}{b} \sum_{j \in \mathcal{B}_m} (Y_j - Q_{\mathbf{w}}(S_j, A_j))^2$$



One DQN Minibatch Update

Sample a minibatch $\mathcal{B}_m$ containing b transitions from the replay buffer.

1. Compute one target for every transition:

$$Y_j = R_{j+1} + \gamma(1 - d_j) \max_{a'} Q_{\mathbf{w}}^t(S_{j+1}, a'), \quad j \in \mathcal{B}_m.$$

2. Get q-value predictions from the online network for every (S_j, A_j) in the batch: $Q_{\mathbf{w}}(S_j, A_j)$
3. Average the squared TD errors:

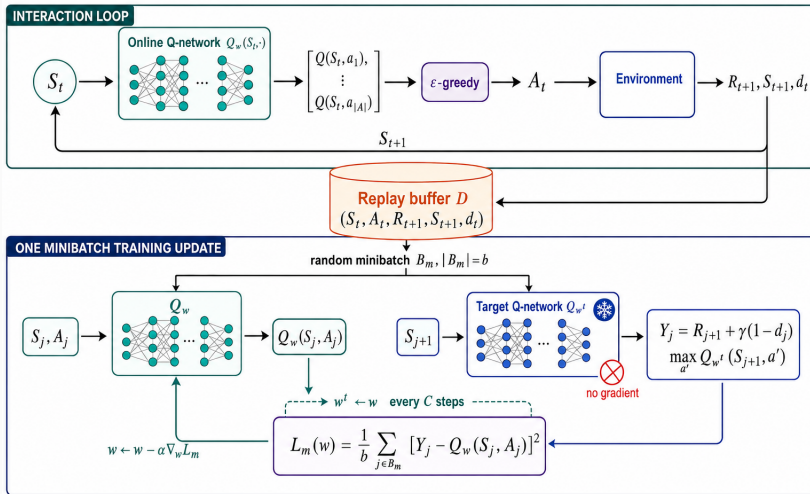
$$L_m(\mathbf{w}) = \frac{1}{b} \sum_{j \in \mathcal{B}_m} (Y_j - Q_{\mathbf{w}}(S_j, A_j))^2$$

4. Update only the online-network parameters:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla_{\mathbf{w}} L_m(\mathbf{w}).$$



Overall DQN Workflow



Deep Deterministic Policy Gradient (DDPG)

Motivation for DDPG

- ▶ DQN selects the action with the largest predicted Q-value: $A_t = \arg \max_{a \in \mathcal{A}} Q_w(S_t, a)$
- ▶ This is practical when $\mathcal{A}$ contains a manageable number of discrete actions because the network can evaluate every action.



Motivation for DDPG

- ▶ DQN selects the action with the largest predicted Q-value: $A_t = \arg \max_{a \in \mathcal{A}} Q_w(S_t, a)$
- ▶ This is practical when $\mathcal{A}$ contains a manageable number of discrete actions because the network can evaluate every action.
- ▶ In continuous control, the action may take infinitely many values.
- ▶ A DQN cannot produce one output for every possible continuous action.
- ▶ Numerically maximising $Q_w(S_t, a)$ over a at every timestep may also be computationally expensive.



Motivation for DDPG

- ▶ DQN selects the action with the largest predicted Q-value: $A_t = \arg \max_{a \in \mathcal{A}} Q_w(S_t, a)$
- ▶ This is practical when $\mathcal{A}$ contains a manageable number of discrete actions because the network can evaluate every action.
- ▶ In continuous control, the action may take infinitely many values.
- ▶ A DQN cannot produce one output for every possible continuous action.
- ▶ Numerically maximising $Q_w(S_t, a)$ over a at every timestep may also be computationally expensive.

DDPG's Solution

Learn a deterministic actor that directly produces a deterministic continuous action:

$$A_t = \mu_{\theta}(S_t) \approx \arg \max_a Q_w(S_t, a).$$

The critic then evaluates the action produced by this actor.



DDPG Combines Actor–Critic Learning with DQN Ideas

DDPG contains an actor, a critic and slowly changing target copies.

Online Networks

Actor: $A = \mu_{\theta}(S)$

Critic: $Q_w(S, A)$

Estimates the expected return from taking action A in state S .



DDPG Combines Actor–Critic Learning with DQN Ideas

DDPG contains an actor, a critic and slowly changing target copies.

Online Networks

Actor: $A = \mu_{\theta}(S)$

Critic: $Q_w(S, A)$

Estimates the expected return from taking action A in state S .

Target Networks

Target actor: $A' = \mu_{\theta}^t(S')$

Target critic: $Q_w^t(S', A')$

Provide slowly changing bootstrap targets for critic training.



DDPG Combines Actor–Critic Learning with DQN Ideas

DDPG contains an actor, a critic and slowly changing target copies.

Online Networks

Actor: $A = \mu_{\theta}(S)$

Critic: $Q_w(S, A)$

Estimates the expected return from taking action A in state S .

Target Networks

Target actor: $A' = \mu_{\theta}^t(S')$

Target critic: $Q_w^t(S', A')$

Provide slowly changing bootstrap targets for critic training.

- A replay buffer stores past transitions for off-policy training.



DDPG Combines Actor–Critic Learning with DQN Ideas

DDPG contains an actor, a critic and slowly changing target copies.

Online Networks

Actor: $A = \mu_{\theta}(S)$

Critic: $Q_w(S, A)$

Estimates the expected return from taking action A in state S .

Target Networks

Target actor: $A' = \mu_{\theta}^t(S')$

Target critic: $Q_w^t(S', A')$

Provide slowly changing bootstrap targets for critic training.

- ▶ A replay buffer stores past transitions for off-policy training.
- ▶ Exploration is introduced by adding noise to the actor's action:

$$A_t = \text{clip}(\mu_{\theta}(S_t) + \eta_t, a_{\min}, a_{\max}).$$

- ▶ Only the noisy action is used for exploration; the actor itself remains deterministic.



DDPG: Critic and Actor Updates

Sample a minibatch $\mathcal{B}_m$ of b transitions from replay buffer $\mathcal{D}$.

1. Critic Update

- ▶ Target actor selects the next action:

$$A'_j = \mu_{\theta^t}(S_{j+1}).$$

- ▶ Target critic computes the TD target:

$$Y_j = R_{j+1} + \gamma(1 - d_j)Q_{\mathbf{w}}^t(S_{j+1}, A'_j).$$



DDPG: Critic and Actor Updates

Sample a minibatch $\mathcal{B}_m$ of b transitions from replay buffer $\mathcal{D}$.

1. Critic Update

- ▶ Target actor selects the next action:

$$A'_j = \mu_{\theta^t}(S_{j+1}).$$

- ▶ Target critic computes the TD target:

$$Y_j = R_{j+1} + \gamma(1 - d_j)Q_{\mathbf{w}}^t(S_{j+1}, A'_j).$$

- ▶ Critic loss:

$$L_m^Q(\mathbf{w}) = \frac{1}{b} \sum_{j \in \mathcal{B}_m} [Y_j - Q_{\mathbf{w}}(S_j, A_j)]^2.$$

- ▶ Update using gradient descent:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha_Q \nabla_{\mathbf{w}} L_m^Q$$



DDPG: Critic and Actor Updates

Sample a minibatch $\mathcal{B}_m$ of b transitions from replay buffer $\mathcal{D}$.

1. Critic Update

- ▶ Target actor selects the next action:

$$A'_j = \mu_{\theta^t}(S_{j+1}).$$

- ▶ Target critic computes the TD target:

$$Y_j = R_{j+1} + \gamma(1 - d_j)Q_{\mathbf{w}}^t(S_{j+1}, A'_j).$$

- ▶ Critic loss:

$$L_m^Q(\mathbf{w}) = \frac{1}{b} \sum_{j \in \mathcal{B}_m} [Y_j - Q_{\mathbf{w}}(S_j, A_j)]^2.$$

- ▶ Update using gradient descent:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha_Q \nabla_{\mathbf{w}} L_m^Q$$

2. Actor Update

- ▶ For every state in the minibatch, the current actor proposes an action: $\tilde{A}_j = \mu_{\theta}(S_j)$

- ▶ The critic evaluates its predicted long-term value:
 $Q_{\mathbf{w}}(S_j, \tilde{A}_j).$



DDPG: Critic and Actor Updates

Sample a minibatch $\mathcal{B}_m$ of b transitions from replay buffer $\mathcal{D}$.

1. Critic Update

- ▶ Target actor selects the next action:

$$A'_j = \mu_{\theta^t}(S_{j+1}).$$

- ▶ Target critic computes the TD target:

$$Y_j = R_{j+1} + \gamma(1 - d_j)Q_w^t(S_{j+1}, A'_j).$$

- ▶ Critic loss:

$$L_m^Q(\mathbf{w}) = \frac{1}{b} \sum_{j \in \mathcal{B}_m} [Y_j - Q_w(S_j, A_j)]^2.$$

- ▶ Update using gradient descent:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha_Q \nabla_{\mathbf{w}} L_m^Q$$

2. Actor Update

- ▶ For every state in the minibatch, the current actor proposes an action: $\tilde{A}_j = \mu_{\theta}(S_j)$

- ▶ The critic evaluates its predicted long-term value:
 $Q_w(S_j, \tilde{A}_j).$

- ▶ The actor maximises the average critic evaluation across all b minibatch transitions:

$$\max_{\theta} J_m(\theta) = \frac{1}{b} \sum_{j \in \mathcal{B}_m} Q_w(S_j, \mu_{\theta}(S_j))$$



DDPG: Critic and Actor Updates

Sample a minibatch $\mathcal{B}_m$ of b transitions from replay buffer $\mathcal{D}$.

1. Critic Update

- ▶ Target actor selects the next action:

$$A'_j = \mu_{\theta^t}(S_{j+1}).$$

- ▶ Target critic computes the TD target:

$$Y_j = R_{j+1} + \gamma(1 - d_j)Q_w^t(S_{j+1}, A'_j).$$

- ▶ Critic loss:

$$L_m^Q(\mathbf{w}) = \frac{1}{b} \sum_{j \in \mathcal{B}_m} [Y_j - Q_w(S_j, A_j)]^2.$$

- ▶ Update using gradient descent:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha_Q \nabla_{\mathbf{w}} L_m^Q$$

2. Actor Update

- ▶ For every state in the minibatch, the current actor proposes an action: $\tilde{A}_j = \mu_{\theta}(S_j)$

- ▶ The critic evaluates its predicted long-term value:
 $Q_w(S_j, \tilde{A}_j).$

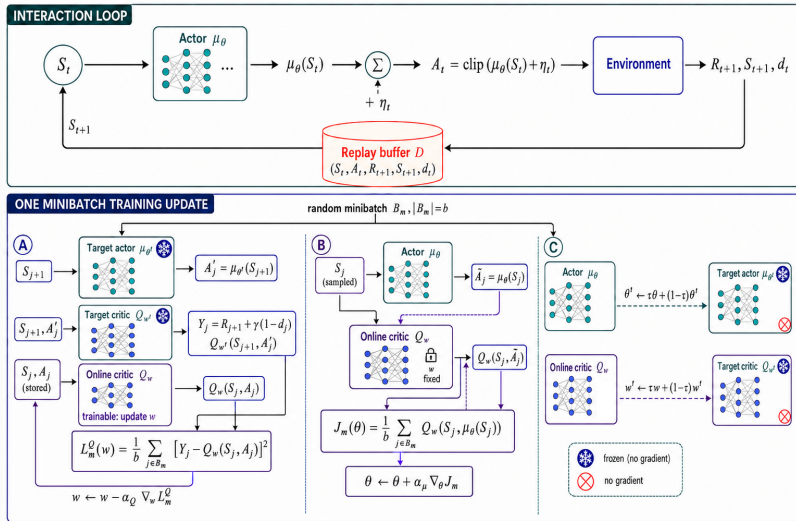
- ▶ The actor maximises the average critic evaluation across all b minibatch transitions:

$$\max_{\theta} J_m(\theta) = \frac{1}{b} \sum_{j \in \mathcal{B}_m} Q_w(S_j, \mu_{\theta}(S_j))$$

- ▶ During this update, $\mathbf{w}$ is fixed and the gradient computed: $\theta \leftarrow \theta + \alpha_{\mu} \nabla_{\theta} J_m$.



Overall DDPG Workflow



Stabilisation and Exploration in DDPG

DDPG combines function approximation, bootstrapping and off-policy learning. Three mechanisms are therefore central to practical training.

1. Experience replay

$$\mathcal{B}_m \sim \mathcal{D}$$

Random minibatches reduce temporal correlation and allow previous experience to be reused.

2. Slow target-network updates

$$\mathbf{w}^t \leftarrow \tau \mathbf{w} + (1 - \tau) \mathbf{w}^t, \quad \boldsymbol{\theta}^t \leftarrow \tau \boldsymbol{\theta} + (1 - \tau) \boldsymbol{\theta}^t,$$

where $\tau \ll 1$. This keeps critic targets from changing abruptly.

3. Explicit exploration noise

$$A_t = \text{clip}(\mu_{\boldsymbol{\theta}}(S_t) + \eta_t, a_{\min}, a_{\max}).$$

The noisy action is executed and stored in the replay buffer.



Decision Transformer (DT)

Motivation for Decision Transformers

- ▶ DQN and DDPG learn using **Bellman targets** and repeatedly interact with the environment.
- ▶ In many industrial applications, online exploration may be costly, unsafe or impractical.
- ▶ However, historical operating data may already contain trajectories with different levels of performance:

$$\mathcal{D} = \left\{ \tau^{(i)} \right\}_{i=1}^N, \quad \tau^{(i)} = (S_1, A_1, R_2, \dots, S_T, A_T, R_{T+1}).$$

- ▶ Can we learn a policy directly from these trajectories without explicitly learning a value function?

Decision Transformer

Represent decision-making as a **sequence-modelling problem**: given the desired return and previous experience, predict the next action.

Desired return + history of states and actions $\longrightarrow$ next action



Decision Transformer: Sequence Representation

For every timestep in an offline trajectory, compute the undiscounted return-to-go:

$$G_t = \sum_{k=t}^T R_{k+1}.$$

The trajectory is rearranged as a sequence of three types of tokens:

$$\underbrace{G_1, S_1, A_1}_{t=1}, \underbrace{G_2, S_2, A_2}_{t=2}, \dots, \underbrace{G_T, S_T, A_T}_{t=T}.$$



Decision Transformer: Sequence Representation

For every timestep in an offline trajectory, compute the undiscounted return-to-go:

$$G_t = \sum_{k=t}^T R_{k+1}.$$

The trajectory is rearranged as a sequence of three types of tokens:

$$\underbrace{G_1, S_1, A_1}_{t=1}, \underbrace{G_2, S_2, A_2}_{t=2}, \dots, \underbrace{G_T, S_T, A_T}_{t=T}.$$

- ▶ G_t : performance still achievable from timestep t .
- ▶ S_t : state encountered by the agent.
- ▶ A_t : action taken in that state.
- ▶ A causal Transformer learns relationships across these tokens while using only the available history.



Decision Transformer – Preparing Training Data

1. Collect Offline Trajectories

$$\mathcal{D} = \{\tau^{(1)}, \tau^{(2)}, \dots, \tau^{(N)}\}, \quad \tau^{(i)} = (S_1, A_1, R_2, \dots, S_T, A_T, R_{T+1}).$$

Trajectories may come from historical logs, simulations, expert policies, random policies or partially trained RL policies.



Decision Transformer – Preparing Training Data

1. Collect Offline Trajectories

$$\mathcal{D} = \{\tau^{(1)}, \tau^{(2)}, \dots, \tau^{(N)}\}, \quad \tau^{(i)} = (S_1, A_1, R_2, \dots, S_T, A_T, R_{T+1}).$$

Trajectories may come from historical logs, simulations, expert policies, random policies or partially trained RL policies.

2. Compute Return-to-Go

$$G_t = \sum_{k=t}^T R_{k+1}.$$

This produces triples (G_t, S_t, A_t) at every timestep.



Decision Transformer – Preparing Training Data

1. Collect Offline Trajectories

$$\mathcal{D} = \{\tau^{(1)}, \tau^{(2)}, \dots, \tau^{(N)}\}, \quad \tau^{(i)} = (S_1, A_1, R_2, \dots, S_T, A_T, R_{T+1}).$$

Trajectories may come from historical logs, simulations, expert policies, random policies or partially trained RL policies.

2. Compute Return-to-Go

$$G_t = \sum_{k=t}^T R_{k+1}.$$

This produces triples (G_t, S_t, A_t) at every timestep.

3. Create a Training Minibatch

- Sample B trajectory subsequences, each containing at most K timesteps.
- Example: $B = 32$, $K = 20$ gives at most 640 action targets.
- Shorter subsequences are padded; padded positions are excluded from the loss.
- Longer sequences are broken down into sub-sequences.



Decision Transformer – Preparing Training Data

1. Collect Offline Trajectories

$$\mathcal{D} = \{\tau^{(1)}, \tau^{(2)}, \dots, \tau^{(N)}\}, \quad \tau^{(i)} = (S_1, A_1, R_2, \dots, S_T, A_T, R_{T+1}).$$

Trajectories may come from historical logs, simulations, expert policies, random policies or partially trained RL policies.

2. Compute Return-to-Go

$$G_t = \sum_{k=t}^T R_{k+1}.$$

This produces triples (G_t, S_t, A_t) at every timestep.

3. Create a Training Minibatch

- Sample B trajectory subsequences, each containing at most K timesteps.
- Example: $B = 32$, $K = 20$ gives at most 640 action targets.
- Shorter subsequences are padded; padded positions are excluded from the loss.
- Longer sequences are broken down into sub-sequences.

4. Build One Token Sequence per Subsequence

$$[G_{i,1}, S_{i,1}, A_{i,1}, \dots, G_{i,K}, S_{i,K}, A_{i,K}], \quad i = 1, \dots, B.$$



Decision Transformer – Learning from a Minibatch

5. Embed the Tokens

- Map each return, state and action to a d -dimensional vector and add timestep embedding:

$$\mathbf{z}_{i,k}^G, \mathbf{z}_{i,k}^S, \mathbf{z}_{i,k}^A \in \mathbb{R}^d.$$



Decision Transformer – Learning from a Minibatch

5. Embed the Tokens

- Map each return, state and action to a d -dimensional vector and add timestep embedding:

$$\mathbf{z}_{i,k}^G, \mathbf{z}_{i,k}^S, \mathbf{z}_{i,k}^A \in \mathbb{R}^d.$$

6. Predict Actions Causally

- The causal Transformer produces contextualised token representations.
- The output at each state-token position predicts its corresponding action:

$$\hat{A}_{i,k} = \text{ActionHead}(\tilde{\mathbf{z}}_{i,k}^S).$$

Because $A_{i,k}$ appears after $S_{i,k}$, the prediction cannot use the action it is trying to predict.



Decision Transformer – Learning from a Minibatch

5. Embed the Tokens

- Map each return, state and action to a d -dimensional vector and add timestep embedding:

$$\mathbf{z}_{i,k}^G, \mathbf{z}_{i,k}^S, \mathbf{z}_{i,k}^A \in \mathbb{R}^d.$$

6. Predict Actions Causally

- The causal Transformer produces contextualised token representations.
- The output at each state-token position predicts its corresponding action:

$$\hat{A}_{i,k} = \text{ActionHead}(\tilde{\mathbf{z}}_{i,k}^S).$$

Because $A_{i,k}$ appears after $S_{i,k}$, the prediction cannot use the action it is trying to predict.

7. Compute the Minibatch Loss

$$\mathcal{L}(\theta) = \frac{1}{N_{\text{valid}}} \sum_{i=1}^B \sum_{k=1}^K m_{i,k} \ell(A_{i,k}, \hat{A}_{i,k}),$$

where $m_{i,k} = 1$ for a valid timestep and 0 for padding.

- Use MSE for continuous actions and cross-entropy for discrete actions.
- Update the model once, sample another minibatch and repeat.



Converting Return, State and Action into Vector Embeddings

Example: Industrial valve control

$$G_t = 120, \quad S_t = \begin{bmatrix} 82 \\ 3.1 \\ 0.65 \end{bmatrix}, \quad A_t = [0.40]$$

82 : temperature ($^{\circ}\text{C}$)

3.1 : pressure (bar)

0.65 : liquid level

0.40 : valve opening



Converting Return, State and Action into Vector Embeddings

Example: Industrial valve control

$$G_t = 120, \quad S_t = \begin{bmatrix} 82 \\ 3.1 \\ 0.65 \end{bmatrix}, \quad A_t = [0.40]$$

82 : temperature ($^{\circ}\text{C}$)

3.1 : pressure (bar)

0.65 : liquid level

0.40 : valve opening

The inputs have different dimensions:

$$G_t \in \mathbb{R}, \quad S_t \in \mathbb{R}^3, \quad A_t \in \mathbb{R}.$$



Converting Return, State and Action into Vector Embeddings

Example: Industrial valve control

$$G_t = 120, \quad S_t = \begin{bmatrix} 82 \\ 3.1 \\ 0.65 \end{bmatrix}, \quad A_t = [0.40]$$

82 : temperature ($^{\circ}\text{C}$)

3.1 : pressure (bar)

0.65 : liquid level

0.40 : valve opening

The inputs have different dimensions:

$$G_t \in \mathbb{R}, \quad S_t \in \mathbb{R}^3, \quad A_t \in \mathbb{R}.$$

Learned Projections:

$$\mathbf{e}_t^G = \mathbf{W}_G G_t + \mathbf{b}_G$$

$$\mathbf{e}_t^S = \mathbf{W}_S S_t + \mathbf{b}_S$$

$$\mathbf{e}_t^A = \mathbf{W}_A A_t + \mathbf{b}_A$$

Each projection produces a vector of the same size:

$$\mathbf{e}_t^G, \mathbf{e}_t^S, \mathbf{e}_t^A \in \mathbb{R}^d.$$



Converting Return, State and Action into Vector Embeddings

Example: Industrial valve control

$$G_t = 120, \quad S_t = \begin{bmatrix} 82 \\ 3.1 \\ 0.65 \end{bmatrix}, \quad A_t = [0.40]$$

82 : temperature ($^{\circ}\text{C}$)

3.1 : pressure (bar)

0.65 : liquid level

0.40 : valve opening

The inputs have different dimensions:

$$G_t \in \mathbb{R}, \quad S_t \in \mathbb{R}^3, \quad A_t \in \mathbb{R}.$$

Learned Projections:

$$\mathbf{e}_t^G = \mathbf{W}_G G_t + \mathbf{b}_G$$

$$\mathbf{e}_t^S = \mathbf{W}_S S_t + \mathbf{b}_S$$

$$\mathbf{e}_t^A = \mathbf{W}_A A_t + \mathbf{b}_A$$

Each projection produces a vector of the same size:

$$\mathbf{e}_t^G, \mathbf{e}_t^S, \mathbf{e}_t^A \in \mathbb{R}^d.$$

Add the learned timestep embedding $\mathbf{p}_t$:

$$\mathbf{z}_t^G = \mathbf{e}_t^G + \mathbf{p}_t, \quad \mathbf{z}_t^S = \mathbf{e}_t^S + \mathbf{p}_t, \quad \mathbf{z}_t^A = \mathbf{e}_t^A + \mathbf{p}_t.$$



Converting Return, State and Action into Vector Embeddings

Example: Industrial valve control

$$G_t = 120, \quad S_t = \begin{bmatrix} 82 \\ 3.1 \\ 0.65 \end{bmatrix}, \quad A_t = [0.40]$$

82 : temperature ($^{\circ}\text{C}$)

3.1 : pressure (bar)

0.65 : liquid level

0.40 : valve opening

The inputs have different dimensions:

$$G_t \in \mathbb{R}, \quad S_t \in \mathbb{R}^3, \quad A_t \in \mathbb{R}.$$

Learned Projections:

$$\mathbf{e}_t^G = \mathbf{W}_G G_t + \mathbf{b}_G$$

$$\mathbf{e}_t^S = \mathbf{W}_S S_t + \mathbf{b}_S$$

$$\mathbf{e}_t^A = \mathbf{W}_A A_t + \mathbf{b}_A$$

Each projection produces a vector of the same size:

$$\mathbf{e}_t^G, \mathbf{e}_t^S, \mathbf{e}_t^A \in \mathbb{R}^d.$$

Add the learned timestep embedding $\mathbf{p}_t$:

$$\mathbf{z}_t^G = \mathbf{e}_t^G + \mathbf{p}_t, \quad \mathbf{z}_t^S = \mathbf{e}_t^S + \mathbf{p}_t, \quad \mathbf{z}_t^A = \mathbf{e}_t^A + \mathbf{p}_t.$$

Input to the Transformer:

$$\mathbf{z}_1^G, \mathbf{z}_1^S, \mathbf{z}_1^A, \mathbf{z}_2^G, \mathbf{z}_2^S, \mathbf{z}_2^A, \dots$$

Note: Return, state and action remain separate, but all have the same embedding dimension d .



Decision Transformer – Inference

1. Specify the desired return and observe the initial state

$$G_1^{\text{target}} = G_{\text{desired}}, \quad S_1 = \text{initial state.}$$

Choose G_{desired} within a realistic range represented in the training data.



Decision Transformer – Inference

1. Specify the desired return and observe the initial state

$$G_1^{\text{target}} = G_{\text{desired}}, \quad S_1 = \text{initial state.}$$

Choose G_{desired} within a realistic range represented in the training data.

2. Predict the current action: $[G_1^{\text{target}}, S_1, A_1, \dots, G_t^{\text{target}}, S_t] \xrightarrow{\text{Decision Transformer}} \hat{A}_t.$



Decision Transformer – Inference

1. Specify the desired return and observe the initial state

$$G_1^{\text{target}} = G_{\text{desired}}, \quad S_1 = \text{initial state}.$$

Choose G_{desired} within a realistic range represented in the training data.

2. Predict the current action: $[G_1^{\text{target}}, S_1, A_1, \dots, G_t^{\text{target}}, S_t] \xrightarrow{\text{Decision Transformer}} \hat{A}_t.$
3. Execute the action $A_t = \hat{A}_t \longrightarrow \text{Environment} \longrightarrow R_{t+1}, S_{t+1}.$



Decision Transformer – Inference

1. Specify the desired return and observe the initial state

$$G_1^{\text{target}} = G_{\text{desired}}, \quad S_1 = \text{initial state}.$$

Choose G_{desired} within a realistic range represented in the training data.

2. Predict the current action: $[G_1^{\text{target}}, S_1, A_1, \dots, G_t^{\text{target}}, S_t] \xrightarrow{\text{Decision Transformer}} \hat{A}_t.$
3. Execute the action $A_t = \hat{A}_t \longrightarrow \text{Environment} \longrightarrow R_{t+1}, S_{t+1}.$
4. Update the remaining desired return

$$G_{t+1}^{\text{target}} = G_t^{\text{target}} - R_{t+1}.$$

Append $(A_t, G_{t+1}^{\text{target}}, S_{t+1})$ to the sequence and predict the next action.



Decision Transformer – Inference

1. Specify the desired return and observe the initial state

$$G_1^{\text{target}} = G_{\text{desired}}, \quad S_1 = \text{initial state}.$$

Choose G_{desired} within a realistic range represented in the training data.

2. Predict the current action: $[G_1^{\text{target}}, S_1, A_1, \dots, G_t^{\text{target}}, S_t] \xrightarrow{\text{Decision Transformer}} \hat{A}_t.$

3. Execute the action $A_t = \hat{A}_t \longrightarrow \text{Environment} \longrightarrow R_{t+1}, S_{t+1}.$

4. Update the remaining desired return

$$G_{t+1}^{\text{target}} = G_t^{\text{target}} - R_{t+1}.$$

Append $(A_t, G_{t+1}^{\text{target}}, S_{t+1})$ to the sequence and predict the next action.

5. Maintain the context window

- If $t \leq K$, use the complete available history.
- If $t > K$, retain only the most recent K timesteps.



Decision Transformer – Inference

1. Specify the desired return and observe the initial state

$$G_1^{\text{target}} = G_{\text{desired}}, \quad S_1 = \text{initial state.}$$

Choose G_{desired} within a realistic range represented in the training data.

2. Predict the current action: $[G_1^{\text{target}}, S_1, A_1, \dots, G_t^{\text{target}}, S_t] \xrightarrow{\text{Decision Transformer}} \hat{A}_t.$

3. Execute the action $A_t = \hat{A}_t \longrightarrow \text{Environment} \longrightarrow R_{t+1}, S_{t+1}.$

4. Update the remaining desired return

$$G_{t+1}^{\text{target}} = G_t^{\text{target}} - R_{t+1}.$$

Append $(A_t, G_{t+1}^{\text{target}}, S_{t+1})$ to the sequence and predict the next action.

5. Maintain the context window

- If $t \leq K$, use the complete available history.
- If $t > K$, retain only the most recent K timesteps.

Inference Loop

Desired return + recent history $\rightarrow A_t \rightarrow R_{t+1}, S_{t+1} \rightarrow$ updated desired return.



► Advantages:

- Leverages advances in large-scale transformers.
- Naturally handles long horizons and variable-length episodes.
- Simple supervised training involving no bootstrapping or TD backups and avoids the deadly triad



► Advantages:

- Leverages advances in large-scale transformers.
- Naturally handles long horizons and variable-length episodes.
- Simple supervised training involving no bootstrapping or TD backups and avoids the deadly triad

► Challenges:

- High data requirements (large offline datasets).
- Computationally expensive to train.
- Limited exploration at deployment time.
- May struggle with sparse-reward tasks.



Multi-Agent RL

Motivation for Multi-Agent Reinforcement Learning

- So far, we assumed that a **single agent** makes all decisions affecting the environment.



Motivation for Multi-Agent Reinforcement Learning

- ▶ So far, we assumed that a **single agent** makes all decisions affecting the environment.
- ▶ Many real systems contain multiple decision-makers acting at the same time:
 - robots sharing a factory or warehouse;
 - vehicles interacting in traffic;
 - machines competing for energy or production resources;
 - firms, bidders or players pursuing different objectives.



Motivation for Multi-Agent Reinforcement Learning

- ▶ So far, we assumed that a **single agent** makes all decisions affecting the environment.
- ▶ Many real systems contain multiple decision-makers acting at the same time:
 - robots sharing a factory or warehouse;
 - vehicles interacting in traffic;
 - machines competing for energy or production resources;
 - firms, bidders or players pursuing different objectives.
- ▶ The outcome of one agent's action now depends on the actions selected by the other agents.

Agent 1 action + $\dots$ + Agent N action $\longrightarrow$ joint effect on the environment.



Motivation for Multi-Agent Reinforcement Learning

- ▶ So far, we assumed that a **single agent** makes all decisions affecting the environment.
- ▶ Many real systems contain multiple decision-makers acting at the same time:
 - robots sharing a factory or warehouse;
 - vehicles interacting in traffic;
 - machines competing for energy or production resources;
 - firms, bidders or players pursuing different objectives.
- ▶ The outcome of one agent's action now depends on the actions selected by the other agents.

Agent 1 action + $\dots$ + Agent N action $\longrightarrow$ joint effect on the environment.

Multi-Agent Reinforcement Learning

MARL studies how multiple agents learn to cooperate, compete or do both while interacting in a shared environment.



Multi-Agent RL: Interaction and Objective

Consider N agents interacting with a shared environment. At timestep t , agent i :

$$\underbrace{O_t^i}_{\text{local observation}} \xrightarrow{\pi_i} \underbrace{A_t^i}_{\text{individual action}} .$$

- ▶ The environment receives the joint action: $\mathbf{A}_t = (A_t^1, A_t^2, \dots, A_t^N)$
- ▶ Produces the next state $S_{t+1} \sim P(\cdot \mid S_t, \mathbf{A}_t)$
- ▶ Gives rewards: $(R_{t+1}^1, \dots, R_{t+1}^N)$

Types of Interaction among Agents:

- ▶ **Cooperative:** agents share and maximise a team objective
- ▶ **Competitive:** improving one agent's outcome may reduce another's.
- ▶ **Mixed:** agents cooperate on some goals and compete on others.



Cooperative MARL: One Team, Multiple Agents

- ▶ All agents work towards a **common system-level objective**.
- ▶ Each agent may observe only part of the environment and select its own action:

$$O_t^i \xrightarrow{\pi_i} A_t^i.$$



Cooperative MARL: One Team, Multiple Agents

- ▶ All agents work towards a **common system-level objective**.
- ▶ Each agent may observe only part of the environment and select its own action:

$$O_t^i \xrightarrow{\pi_i} A_t^i.$$

- ▶ Their joint action determines the next state and a shared reward:

$$\mathbf{A}_t = (A_t^1, \dots, A_t^N) \longrightarrow S_{t+1}, R_{t+1}^{\text{team}}.$$



Cooperative MARL: One Team, Multiple Agents

- ▶ All agents work towards a **common system-level objective**.
- ▶ Each agent may observe only part of the environment and select its own action:

$$O_t^i \xrightarrow{\pi_i} A_t^i.$$

- ▶ Their joint action determines the next state and a shared reward:

$$\mathbf{A}_t = (A_t^1, \dots, A_t^N) \longrightarrow S_{t+1}, R_{t+1}^{\text{team}}.$$

- ▶ The policies are learned to maximise the team return:

$$J_{\text{team}} = \mathbb{E} \left[\sum_{t=0}^{T-1} \gamma^t R_{t+1}^{\text{team}} \right].$$



Cooperative MARL: One Team, Multiple Agents

- ▶ All agents work towards a **common system-level objective**.
- ▶ Each agent may observe only part of the environment and select its own action:

$$O_t^i \xrightarrow{\pi_i} A_t^i.$$

- ▶ Their joint action determines the next state and a shared reward:

$$\mathbf{A}_t = (A_t^1, \dots, A_t^N) \longrightarrow S_{t+1}, R_{t+1}^{\text{team}}.$$

- ▶ The policies are learned to maximise the team return:

$$J_{\text{team}} = \mathbb{E} \left[\sum_{t=0}^{T-1} \gamma^t R_{t+1}^{\text{team}} \right].$$

Core Intuition

An action may be inconvenient for one agent but still be desirable if it improves the overall performance of the team.

Examples

Warehouse robots, coordinated traffic signals, drone teams and distributed energy-management systems.



Cooperative Example: Warehouse Robots

Problem Setup

- ▶ Several robots move products from storage racks to packing stations.
- ▶ Robots share narrow aisles and charging stations.
- ▶ Each robot observes nearby obstacles, robots and its assigned destination.
- ▶ Actions include move, turn, wait and recharge.



Cooperative Example: Warehouse Robots

Problem Setup

- ▶ Several robots move products from storage racks to packing stations.
- ▶ Robots share narrow aisles and charging stations.
- ▶ Each robot observes nearby obstacles, robots and its assigned destination.
- ▶ Actions include move, turn, wait and recharge.

Illustrative Team Reward

$$R_{t+1}^{\text{team}} = 10D_{t+1} - 5C_{t+1} - 0.1W_{t+1},$$

where

D_{t+1} : completed deliveries,
 C_{t+1} : collisions or blockages,
 W_{t+1} : total waiting time.



Cooperative Example: Warehouse Robots

Problem Setup

- ▶ Several robots move products from storage racks to packing stations.
- ▶ Robots share narrow aisles and charging stations.
- ▶ Each robot observes nearby obstacles, robots and its assigned destination.
- ▶ Actions include move, turn, wait and recharge.

Coordination Intuition

The shortest route for every individual robot may create congestion. One robot waiting or taking a longer route can reduce the total delivery time of the entire fleet.

Illustrative Team Reward

$$R_{t+1}^{\text{team}} = 10D_{t+1} - 5C_{t+1} - 0.1W_{t+1},$$

where

- D_{t+1} : completed deliveries,
- C_{t+1} : collisions or blockages,
- W_{t+1} : total waiting time.

Independent shortest routes $\nRightarrow$ best team performance.



Competitive MARL: Learning to Act Against Other Agents

- ▶ Each agent attempts to maximise its own return: $J_i = \mathbb{E} \left[\sum_{t=0}^{T-1} \gamma^t R_{t+1}^i \right]$
- ▶ An improvement in one agent's outcome may reduce another agent's outcome.



Competitive MARL: Learning to Act Against Other Agents

- ▶ Each agent attempts to maximise its own return: $J_i = \mathbb{E} \left[\sum_{t=0}^{T-1} \gamma^t R_{t+1}^i \right]$
- ▶ An improvement in one agent's outcome may reduce another agent's outcome.
- ▶ During training, the agent experiences different actions and strategies from competing agents.
- ▶ It learns how its own actions should change in response to the behaviour of its opponents.



Competitive MARL: Learning to Act Against Other Agents

- ▶ Each agent attempts to maximise its own return: $J_i = \mathbb{E} \left[\sum_{t=0}^{T-1} \gamma^t R_{t+1}^i \right]$
- ▶ An improvement in one agent's outcome may reduce another agent's outcome.
- ▶ During training, the agent experiences different actions and strategies from competing agents.
- ▶ It learns how its own actions should change in response to the behaviour of its opponents.

Important Intuition

The objective is not merely to perform well against one fixed training opponent, but to learn behaviour that remains effective against a range of plausible opponent strategies.



Competitive MARL: Learning to Act Against Other Agents

- ▶ Each agent attempts to maximise its own return: $J_i = \mathbb{E} \left[\sum_{t=0}^{T-1} \gamma^t R_{t+1}^i \right]$
- ▶ An improvement in one agent's outcome may reduce another agent's outcome.
- ▶ During training, the agent experiences different actions and strategies from competing agents.
- ▶ It learns how its own actions should change in response to the behaviour of its opponents.

Important Intuition

The objective is not merely to perform well against one fixed training opponent, but to learn behaviour that remains effective against a range of plausible opponent strategies.

Ultimate Objective

Learn a policy that can make effective decisions during inference when other agents are independently pursuing conflicting objectives.



Competitive MARL: Learning to Act Against Other Agents

- ▶ Each agent attempts to maximise its own return: $J_i = \mathbb{E} \left[\sum_{t=0}^{T-1} \gamma^t R_{t+1}^i \right]$
- ▶ An improvement in one agent's outcome may reduce another agent's outcome.
- ▶ During training, the agent experiences different actions and strategies from competing agents.
- ▶ It learns how its own actions should change in response to the behaviour of its opponents.

Important Intuition

The objective is not merely to perform well against one fixed training opponent, but to learn behaviour that remains effective against a range of plausible opponent strategies.

Ultimate Objective

Learn a policy that can make effective decisions during inference when other agents are independently pursuing conflicting objectives.

Examples

Cybersecurity attacker–defender simulations, competitive bidding, robot games, etc.



Competitive Example: Cybersecurity Attacker and Defender

Attacker

- ▶ Observes discovered system information.
- ▶ Chooses to probe, exploit or move between systems.
- ▶ Tries to access protected assets while avoiding detection.



Competitive Example: Cybersecurity Attacker and Defender

Attacker

- ▶ Observes discovered system information.
- ▶ Chooses to probe, exploit or move between systems.
- ▶ Tries to access protected assets while avoiding detection.

Defender

- ▶ Observes alerts and system behaviour.
- ▶ Chooses to monitor, patch, isolate or restore systems.
- ▶ Tries to prevent compromise while limiting disruption.



Competitive Example: Cybersecurity Attacker and Defender

Attacker

- Observes discovered system information.
- Chooses to probe, exploit or move between systems.
- Tries to access protected assets while avoiding detection.

Defender

- Observes alerts and system behaviour.
- Chooses to monitor, patch, isolate or restore systems.
- Tries to prevent compromise while limiting disruption.

For a simplified zero-sum simulation:

$$R_{t+1}^{\text{defender}} = -R_{t+1}^{\text{attacker}}.$$

Outcome	R^{defender}	R^{attacker}
Attack detected and blocked	+1	-1
Protected asset compromised	-1	+1



Competitive Example: Cybersecurity Attacker and Defender

Attacker

- Observes discovered system information.
- Chooses to probe, exploit or move between systems.
- Tries to access protected assets while avoiding detection.

Defender

- Observes alerts and system behaviour.
- Chooses to monitor, patch, isolate or restore systems.
- Tries to prevent compromise while limiting disruption.

For a simplified zero-sum simulation:

$$R_{t+1}^{\text{defender}} = -R_{t+1}^{\text{attacker}}.$$

Outcome	R^{defender}	R^{attacker}
Attack detected and blocked	+1	-1
Protected asset compromised	-1	+1

Competitive Intuition

A fixed defence strategy can eventually be exploited. Both agents must learn policies that remain effective against changing opponent behaviour.



Mixed MARL: Cooperation and Competition Together

- ▶ Agents share some system-level goals but also pursue individual objectives.
- ▶ Each agent's reward may contain both shared and individual components:

$$R_{t+1}^i = \lambda R_{t+1}^{\text{shared}} + (1 - \lambda) R_{t+1}^{i, \text{individual}}, \quad 0 \leq \lambda \leq 1.$$



Mixed MARL: Cooperation and Competition Together

- ▶ Agents share some system-level goals but also pursue individual objectives.
- ▶ Each agent's reward may contain both shared and individual components:

$$R_{t+1}^i = \lambda R_{t+1}^{\text{shared}} + (1 - \lambda) R_{t+1}^{i, \text{individual}}, \quad 0 \leq \lambda \leq 1.$$

- ▶ Agents may cooperate in one situation and compete in another.
- ▶ The desired behaviour must balance system performance with individual benefit.



Mixed MARL: Cooperation and Competition Together

- ▶ Agents share some system-level goals but also pursue individual objectives.
- ▶ Each agent's reward may contain both shared and individual components:

$$R_{t+1}^i = \lambda R_{t+1}^{\text{shared}} + (1 - \lambda) R_{t+1}^{i, \text{individual}}, \quad 0 \leq \lambda \leq 1.$$

- ▶ Agents may cooperate in one situation and compete in another.
- ▶ The desired behaviour must balance system performance with individual benefit.

Core Intuition

Agents cooperate when their interests align, but negotiate or compete when the same resource cannot be allocated to everyone.



Mixed MARL: Cooperation and Competition Together

- ▶ Agents share some system-level goals but also pursue individual objectives.
- ▶ Each agent's reward may contain both shared and individual components:

$$R_{t+1}^i = \lambda R_{t+1}^{\text{shared}} + (1 - \lambda) R_{t+1}^{i, \text{individual}}, \quad 0 \leq \lambda \leq 1.$$

- ▶ Agents may cooperate in one situation and compete in another.
- ▶ The desired behaviour must balance system performance with individual benefit.

Core Intuition

Agents cooperate when their interests align, but negotiate or compete when the same resource cannot be allocated to everyone.

Examples

Autonomous driving, supply-chain networks and communication systems sharing limited bandwidth.



Mixed Example: Autonomous Vehicles Merging

Problem Setup

- ▶ Multiple vehicles approach a lane-merging region.
- ▶ Each vehicle wants to reach its destination quickly.
- ▶ All vehicles benefit from avoiding collisions and unstable traffic.
- ▶ Actions include accelerate, brake, maintain speed and change lane.



Mixed Example: Autonomous Vehicles Merging

Problem Setup

- ▶ Multiple vehicles approach a lane-merging region.
- ▶ Each vehicle wants to reach its destination quickly.
- ▶ All vehicles benefit from avoiding collisions and unstable traffic.
- ▶ Actions include accelerate, brake, maintain speed and change lane.

Illustrative Reward for Vehicle i

$$R_{t+1}^i = \underbrace{\alpha P_{t+1}^i}_{\text{progress}} - \underbrace{\beta C_{t+1}}_{\text{collision risk}} - \underbrace{\delta D_{t+1}^i}_{\text{delay}},$$

where C_{t+1} reflects a shared safety consequence.



Mixed Example: Autonomous Vehicles Merging

Problem Setup

- ▶ Multiple vehicles approach a lane-merging region.
- ▶ Each vehicle wants to reach its destination quickly.
- ▶ All vehicles benefit from avoiding collisions and unstable traffic.
- ▶ Actions include accelerate, brake, maintain speed and change lane.
 - ▶ Vehicles **cooperate** by maintaining safe separation and avoiding collisions.
 - ▶ They **compete** for limited road space and the opportunity to merge first.

Illustrative Reward for Vehicle i

$$R_{t+1}^i = \underbrace{\alpha P_{t+1}^i}_{\text{progress}} - \underbrace{\beta C_{t+1}}_{\text{collision risk}} - \underbrace{\delta D_{t+1}^i}_{\text{delay}},$$

where C_{t+1} reflects a shared safety consequence.



Mixed Example: Autonomous Vehicles Merging

Problem Setup

- ▶ Multiple vehicles approach a lane-merging region.
- ▶ Each vehicle wants to reach its destination quickly.
- ▶ All vehicles benefit from avoiding collisions and unstable traffic.
- ▶ Actions include accelerate, brake, maintain speed and change lane.
 - ▶ Vehicles **cooperate** by maintaining safe separation and avoiding collisions.
 - ▶ They **compete** for limited road space and the opportunity to merge first.

Mixed-System Intuition

A vehicle may temporarily slow down and allow another vehicle to merge, improving safety and traffic flow, even though it experiences a small delay.

Illustrative Reward for Vehicle i

$$R_{t+1}^i = \underbrace{\alpha P_{t+1}^i}_{\text{progress}} - \underbrace{\beta C_{t+1}}_{\text{collision risk}} - \underbrace{\delta D_{t+1}^i}_{\text{delay}},$$

where C_{t+1} reflects a shared safety consequence.



Multi-Agent RL: Directions for Further Study

Further study can cover:

- ▶ **Game-theoretic foundations:** best responses, Nash equilibrium and other notions of stable multi-agent behaviour.



Multi-Agent RL: Directions for Further Study

Further study can cover:

- ▶ **Game-theoretic foundations:** best responses, Nash equilibrium and other notions of stable multi-agent behaviour.
- ▶ **Centralised training with decentralised execution:** using global information during training while allowing agents to act using local observations during inference.



Multi-Agent RL: Directions for Further Study

Further study can cover:

- ▶ **Game-theoretic foundations:** best responses, Nash equilibrium and other notions of stable multi-agent behaviour.
- ▶ **Centralised training with decentralised execution:** using global information during training while allowing agents to act using local observations during inference.
- ▶ **MARL algorithm families:** Multi-agent actor–critic methods.



Multi-Agent RL: Directions for Further Study

Further study can cover:

- ▶ **Game-theoretic foundations:** best responses, Nash equilibrium and other notions of stable multi-agent behaviour.
- ▶ **Centralised training with decentralised execution:** using global information during training while allowing agents to act using local observations during inference.
- ▶ **MARL algorithm families:** Multi-agent actor–critic methods.
- ▶ **Coordination and communication:** learning when agents should share information, divide responsibilities or form specialised roles.
- ▶ **Opponent learning:** self-play, opponent modelling and policies that generalise to previously unseen opponents.
- ▶ **Practical deployment:** scalability, safety, robustness, communication failures and transfer from simulation to real systems.



Foundations

- ▶ S. Albrecht, F. Christianos and L. Schäfer, [Multi-Agent Reinforcement Learning: Foundations and Modern Approaches](#).
Free book, lecture slides, exercises and Python code.
- ▶ P. Hernandez-Leal, B. Kartal and M. Taylor, [A Survey and Critique of Multiagent Deep Reinforcement Learning](#).

Hands-On Resources

- ▶ [PettingZoo](#): standard Python environments and APIs for MARL.
- ▶ [RLlib Multi-Agent Environments](#): scalable multi-agent training support.



Deep RL Methods: Key Takeaways

- **DQN** combines Q-learning with deep neural networks to handle large state spaces and discrete actions; replay memory and a target network improve stability.



Deep RL Methods: Key Takeaways

- ▶ **DQN** combines Q-learning with deep neural networks to handle large state spaces and discrete actions; replay memory and a target network improve stability.
- ▶ **DDPG** combines ideas from DQN and actor–critic methods: a deterministic actor selects continuous actions and a critic evaluates them.



Deep RL Methods: Key Takeaways

- ▶ **DQN** combines Q-learning with deep neural networks to handle large state spaces and discrete actions; replay memory and a target network improve stability.
- ▶ **DDPG** combines ideas from DQN and actor–critic methods: a deterministic actor selects continuous actions and a critic evaluates them.
- ▶ **Decision Transformer** learns from offline trajectories by treating decision-making as conditional sequence modelling:
$$\text{desired return} + \text{trajectory history} \longrightarrow \text{action}.$$
- ▶ Unlike DQN and DDPG, Decision Transformer does not require an explicit value function or policy-gradient update.



Deep RL Methods: Key Takeaways

- ▶ **DQN** combines Q-learning with deep neural networks to handle large state spaces and discrete actions; replay memory and a target network improve stability.
- ▶ **DDPG** combines ideas from DQN and actor–critic methods: a deterministic actor selects continuous actions and a critic evaluates them.
- ▶ **Decision Transformer** learns from offline trajectories by treating decision-making as conditional sequence modelling:
$$\text{desired return} + \text{trajectory history} \longrightarrow \text{action}.$$
- ▶ Unlike DQN and DDPG, Decision Transformer does not require an explicit value function or policy-gradient update.
- ▶ **MARL** extends the decision-making setting to multiple interacting agents that may cooperate, compete or do both.
- ▶ The appropriate formulation depends on the action space, available interaction data, safety of exploration and number of decision-makers.

